

Belief-Driven Planning Architecture for LLM Agents

Project Reference Board (Version 0.1)

1. Project Summary

Vision

Develop a structured multi-agent architecture that combines:

- Classical BDI (Belief-Desire-Intention) concepts
- Automated Planning algorithms
- Modern LLM-based reasoning
- Agentic AI execution systems

The goal is to reduce over-reliance on natural language planning and provide a more explainable, modular, and deterministic agent framework.

Core Problem

Current agent frameworks often rely on an LLM supervisor to:

- Understand user requests
- Create plans
- Select agents
- Decide execution order

This creates issues:

- Non-deterministic planning
 - Difficult debugging
 - Limited explainability
 - Poor scalability with many agents
 - Hard-to-analyze decision processes
-

Proposed Solution

Introduce a Belief-Driven Planning Layer between user requests and agent execution.

The architecture separates:

1. Understanding (LLM)
 2. Planning (Algorithms)
 3. Execution (Agents)
 4. State Management (Belief System)
-

2. Architecture Overview

User Goal ↓ Goal Interpreter (LLM) ↓ Structured Objectives ↓ Planner ↓ Goals ↓ Intention Assignment ↓ Agents ↓ Execution Results ↓ Belief Updates ↓ Replanning (if needed)

3. Finalized Components

3.1 Belief Layer

Permanent Belief Base

Purpose: Store persistent system knowledge.

Examples:

- Available agents
- Available tools
- MCP servers
- Resource descriptions
- Capability descriptions
- Policies
- Constraints

Example:

"ResearchAgent can search academic papers."

"VenueAgent can search event venues."

Characteristics:

- Persistent
 - Shared across tasks
 - Loaded at startup
-

Instance Belief Base

Purpose: Store task-specific knowledge.

Examples:

"Venue identified."

"Budget exceeds £50,000."

"Three papers have been collected."

Characteristics:

- Created per task
 - Updated during execution
 - Deleted or archived after completion
-

Belief Format

Initial format:

JSON

Belief contents:

- id
- type
- statement
- source
- confidence
- metadata

Future support:

- Text
 - Images
 - URLs
 - Documents
 - Knowledge resources
-

3.2 Goals & Desires

Desire

Represents the final outcome requested by the user.

Example:

"Plan a 3-day AI conference."

Goal

Structured objective generated from the desire.

Example:

- Budget Approved
- Venue Booked
- Speakers Confirmed
- Schedule Created

Goals become planner inputs.

3.3 Intentions

Intentions represent executable commitments.

Responsibilities:

- Receive planned goals
- Select capable agents
- Execute tasks
- Return outcomes

Current implementation approach:

LLM-based agent selection and tool calling.

No modifications planned initially.

4. Planning System

Final Decision

Planning should NOT be performed directly by the LLM.

LLMs will only:

- Interpret natural language goals
- Convert goals into structured objectives

Planning itself will be algorithm-driven.

Planning Architecture

Natural Language Goal ↓ Goal Interpreter (LLM) ↓ Structured Objective Graph ↓ Planner ↓ Execution Plan

Selected Planning Strategy

Current choice:

Backward Chaining Planner

Reasons:

- Explainable
- Easy to visualize
- Strong theoretical foundation
- Natural fit for goal completion systems

Example:

conference_ready ← schedule_ready ← speakers_confirmed ← venue_booked ← budget_approved

Generated plan:

1. budget_approved
 2. venue_booked
 3. speakers_confirmed
 4. schedule_ready
 5. conference_ready
-

5. Demonstration Use Case

Selected Showcase

AI Conference Planning System

Reason:

Demonstrates planning better than document generation tasks.

User Request

"Plan a 3-day AI conference in Manchester for 500 attendees with a budget of £50,000."

Agents

VenueAgent

Capabilities:

- Search venues
 - Estimate venue cost
-

SpeakerAgent

Capabilities:

- Find speakers
 - Estimate speaker cost
-

BudgetAgent

Capabilities:

- Budget calculation
 - Budget validation
-

ScheduleAgent

Capabilities:

- Event scheduling
 - Agenda generation
-

Example Planning Chain

conference_ready

Depends on:

- schedule_ready

Depends on:

- speakers_confirmed

Depends on:

- venue_booked

Depends on:

- budget_approved
-

6. MVP Scope

Included

- Belief Manager
 - Goal Interpreter
 - Planner
 - Intention Assignment
 - Mock Agents
 - Belief Updates
-

Excluded

- MCP Integration
- LangGraph Integration
- External APIs
- Real-world booking systems
- Production deployment

These will be added later.

7. Proposed Tech Stack

Backend

- Python

Framework

- FastAPI

LLM Layer

- OpenAI
- Claude
- Local models (future)

Planning Layer

- Custom Python implementation

Storage

- JSON (initial)
- SQLite (later)
- PostgreSQL (future)

Agent Communication

- Internal event bus (initial)
- A2A/MCP (future)

Visualization

- Streamlit or React dashboard
-

8. Success Criteria

The system should:

1. Generate structured goals from natural language.
 2. Produce explainable plans.
 3. Assign tasks to appropriate agents.
 4. Update beliefs after execution.
 5. Replan when beliefs change.
 6. Maintain separation between permanent and instance beliefs.
-

9. Roles & Responsibilities

Your Responsibilities

Architecture Owner

Research

Tasks:

- Define belief schema
- Define knowledge representation
- Design belief manager
- Design JSON structures
- Define agent capabilities
- Research planning algorithms
- Prepare publication direction
- Evaluate novelty

Deliverables:

- Belief specifications
 - Architecture diagrams
 - Research positioning
 - Benchmark design
-

AI Responsibilities

System Architect & Technical Co-Designer

Tasks:

- Refine architecture
- Design planner
- Design goal interpreter
- Define interfaces
- Design schemas
- Generate pseudocode
- Review architectural decisions
- Help create experiments
- Assist with publication structure

Deliverables:

- Technical specifications
 - Planning logic
 - Data models
 - Evaluation framework
 - Research paper drafts
-

10. Immediate Next Steps

Priority 1

Define JSON schema for:

- Permanent beliefs
 - Instance beliefs
 - Goals
 - Intentions
-

Priority 2

Build Goal Interpreter

Natural Language → Structured Objectives

Priority 3

Implement Backward Chaining Planner

Objectives → Goal Dependency Graph → Execution Plan

Priority 4

Build Conference Planning Demo

VenueAgent SpeakerAgent BudgetAgent ScheduleAgent

Priority 5

Create Evaluation Metrics

- Goal completion rate
 - Planning accuracy
 - Replanning frequency
 - Token usage
 - Execution cost
 - Explainability score
-

Status: Architecture Definition Complete Next Milestone: Schema Design & Planner Prototype